

FastAPI Cheat Sheet

20 tables · 219 concepts · Updated 2026-09-27 · [cheatgrid.com](#)

FastAPI is a Python web framework for building APIs in which standard Python type hints on your functions drive request parsing, validation and docs. It gets its speed from Starlette and Pydantic, is based on the OpenAPI and JSON Schema standards, and serves automatic interactive docs at `/docs` and `/redoc`. Releases in 2026 serialize JSON responses with Pydantic's Rust core whenever a response model or return type is declared, and added built-in streaming for JSON Lines and Server-Sent Events. The mental model to keep: you declare each type **once**, as a parameter or a return annotation, and that single declaration becomes validation, conversion, editor completion and API documentation.

CONTENTS

Core Application Setup	1
HTTP Methods and Route Definitions	2
Request Parameters	3
Parameter Validation	4
Pydantic Models and Validation	5
Response Configuration	6
Streaming Responses	7
Dependency Injection	8
Error Handling	10
Authentication and Security	11
Middleware	12
Background Tasks	13
Database Integration	13
WebSockets	14
Testing	15
Advanced Routing	16
OpenAPI Customization	17
Settings and Configuration	18
Performance and Deployment	18
Advanced Patterns	19
References	20

TABLE 1

Core Application Setup

Everything starts with the `FastAPI()` instance and the path-operation decorators that hang routes off it. These are the pieces you write in your very first file, creating the app, defining an endpoint, choosing sync or async, and wiring up startup logic with a lifespan handler. The automatic `/docs` and Pydantic integration listed here are also why FastAPI feels productive from line one.

Concept	Example	Description
Install FastAPI (fastapi[standard])	<code>uv add "fastapi[standard]"</code>	- Installs FastAPI with the standard extras (Uvicorn, the <code>fastapi</code> CLI, email validation, Jinja2, etc.) fastapi-slim was dropped, use plain <code>fastapi</code> for a minimal install instead.
FastAPI app instance	<code>app = FastAPI()</code> <code>app = FastAPI(title="My API", version="1.0.0")</code>	- Creates the main application instance - accepts optional metadata like title, description, version for OpenAPI docs.

Concept	Example	Description
Path operation decorator	<pre>@app.get("/") @app.post("/items")</pre>	- Declares an HTTP endpoint for a method and path - matched in declaration order, so a fixed path like <code>/users/me</code> must come before <code>/users/{user_id}</code> or the dynamic route shadows it.
Path operation function	<pre>def read_root(): return {"msg": "Hello"}</pre>	- Function executed when endpoint is hit return value is automatically serialized to JSON.
Run with FastAPI CLI	<pre>fastapi dev main.py fastapi run main.py</pre>	Recommended way to run FastAPI (included in <code>fastapi[standard]</code>) <code>dev</code> for development (auto-reload, binds localhost only); <code>run</code> for production (no reload, binds all interfaces).
Run with Uvicorn	<pre>uvicorn main:app --reload (dev) uvicorn main:app --host 0.0.0.0 --port 80 (manual/production)</pre>	- Installs and runs the ASGI server directly, useful when you want a different ASGI server or flags the <code>fastapi</code> CLI does not expose <code>fastapi dev</code> / <code>fastapi run</code> remain the recommended default, including for Docker deployments.
Debugging with uvicorn.run	<pre>if __name__ == "__main__": uvicorn.run(app, host="0.0.0.0", port=8000)</pre>	- Starts Uvicorn from your own code so an IDE debugger can attach and hit breakpoints - the block only runs when the file is executed directly, not when it is imported.
Async path operation	<pre>async def read_items(): return await db.fetch()</pre>	- Uses <code>async def</code> for non-blocking I/O operations, running directly on the event loop - a plain <code>def</code> function instead runs in an external threadpool, which is why blocking calls belong there, not inside <code>async def</code> .
Lifespan context manager	<pre>from contextlib import asyncontextmanager @asyncontextmanager async def lifespan(app): # startup yield # shutdown app = FastAPI(lifespan=lifespan)</pre>	- Manages startup and shutdown logic in a single function split by <code>yield</code> - replaces the deprecated <code>on_event</code> decorators; setting <code>lifespan</code> disables <code>on_event</code> entirely, it is one or the other, never both.
Automatic documentation	<pre>/docs for Swagger UI /redoc for ReDoc</pre>	- FastAPI auto-generates interactive API docs at these endpoints from your existing type hints and decorators - requires zero configuration, no separate spec file to maintain.
Pydantic integration	<pre>class Item(BaseModel): name: str price: float</pre>	- Pydantic models provide automatic validation and serialization on the request body itself - FastAPI rejects a non-matching body before your function runs, this is separate from <code>response_model</code> , which only shapes the outgoing response.
app.frontend()	<pre>app.frontend("/", directory="dist")</pre>	- Serves a built static frontend (React, Vue, Svelte, etc.) from a directory, checked only when no API route matches - defaults to <code>check_dir="auto"</code> , warning instead of erroring in dev (added in 0.141.0).

TABLE 2

HTTP Methods and Route Definitions

Each HTTP method maps to a decorator and a conventional meaning: GET reads, POST creates, PUT replaces, PATCH partially updates, DELETE removes. Following these conventions keeps your API predictable and lets caches, browsers, and other tools behave correctly. The lesser used HEAD and OPTIONS round out the set for existence checks and CORS handling.

Method	Example	Description
GET request	<pre>@app.get("/items/{id}") def read_item(id: int):</pre>	- Retrieves data no request body, typically uses path or query parameters.
POST request	<pre>@app.post("/items") def create_item(item: Item):</pre>	- Creates new resource - accepts request body defined by Pydantic model.

Method	Example	Description
PUT request	<pre>@app.put("/items/{id}") def update_item(id: int, item: Item):</pre>	- Replaces entire resource - expects complete object in request body.
PATCH request	<pre>@app.patch("/items/{id}") def partial_update(id: int, item: PartialItem):</pre>	- Updates partial resource - only specified fields are modified.
DELETE request	<pre>@app.delete("/items/{id}") def delete_item(id: int):</pre>	- Removes resource - typically returns status confirmation.
HEAD request	<pre>@app.head("/items") def items_head():</pre>	- Returns headers only, no body - useful for checking resource existence or metadata.
OPTIONS request	<pre>@app.options("/items") def items_options():</pre>	- Returns allowed HTTP methods for endpoint - often handled automatically for CORS.
api_route for multiple methods	<pre>@app.api_route("/items", methods=["GET", "POST"]) def handle_items():</pre>	- One function answers several HTTP methods at once <code>methods</code> takes a list of method names.

TABLE 3

Request Parameters

Data reaches your endpoint from several places, the URL path, the query string, the JSON body, headers, cookies, form fields, and uploaded files, and FastAPI pulls each one in just by declaring it as a typed function argument. The standout feature is that the type annotation alone drives parsing and validation, so a path parameter declared as `int` is automatically rejected if someone passes text. Knowing which source maps to which declaration is the core skill this table builds.

Type	Example	Description
Path parameter	<pre>@app.get("/items/{item_id}") def read(item_id: int):</pre>	- Extracts value from URL path type validated automatically based on annotation.
Query parameter	<pre>def read_item(skip: int = 0, limit: int = 10):</pre>	- Reads from URL query string (<code>?skip=0&limit=10</code>) optional with defaults.
Query parameter model	<pre>class FilterParams(BaseModel): limit: int = 100 offset: int = 0 def read_items(filter_query: Annotated[FilterParams, Query()]):</pre>	- Groups related query parameters into one Pydantic model - can <code>forbid</code> extra fields with <code>model_config</code> .
Request body	<pre>def create(item: Item):</pre>	- Accepts JSON payload Pydantic model validates structure and types automatically.
Multiple body parameters	<pre>def create(item: Item, user: User):</pre>	- Accepts multiple JSON objects in request body - FastAPI expects nested structure.
Body(embed=True)	<pre>item: Annotated[Item, Body(embed=True)]</pre>	Wraps a single body parameter under its own key instead of expecting it at the top level.
Header parameter	<pre>from fastapi import Header def read(user_agent: str = Header()):</pre>	- Reads HTTP headers - automatically converts your snake_case parameter name to hyphen-case to match the header.
Cookie parameter	<pre>from fastapi import Cookie def read(session_id: str = Cookie()):</pre>	- Extracts cookie values type-validated like other parameters.
Header/Cookie parameter model	<pre>class CommonHeaders(BaseModel): host: str save_data: bool def read_items(headers: Annotated[CommonHeaders, Header()]):</pre>	Groups related header or cookie values into one Pydantic model, the same technique for both.

Type	Example	Description
Form data	<pre>from fastapi import Form def login(username: str = Form()):</pre>	- Accepts <code>application/x-www-form-urlencoded</code> - requires python-multipart installed.
Form model	<pre>class FormData(BaseModel): username: str password: str def login(data: Annotated[FormData, Form()]):</pre>	Declares a whole group of form fields as one Pydantic model instead of separate <code>Form()</code> parameters.
File upload	<pre>from fastapi import UploadFile def upload(file: UploadFile):</pre>	- Handles file uploads <code>UploadFile</code> provides async methods and metadata like filename.
Multiple file upload	<pre>def upload(files: list[UploadFile]):</pre>	- Accepts multiple files in single request - validated as list.
Using the Request object directly	<pre>from fastapi import Request def read_root(item_id: str, request: Request): client_host = request.client.host</pre>	- Gives raw access to the underlying Starlette request - data read this way is not validated or documented by FastAPI
strict_content_type	<pre>app = FastAPI(strict_content_type=False)</pre>	- By default JSON bodies require a valid Content-Type header - disabling this restores older, looser parsing (0.132.0).

TABLE 4

Parameter Validation

Beyond checking types, you often need real constraints too: a minimum length, a numeric range, a pattern, a required versus optional rule. The modern `Annotated` syntax shown in the first rows is the recommended way to attach these rules while keeping the type and its metadata together. Push validation this far up the stack, and your endpoint body can trust the data is already clean.

Technique	Example	Description
Annotated query validation	<pre>from typing import Annotated q: Annotated[str None, Query(min_length=3)] = None</pre>	Recommended way to use Query, Path, Header, Cookie - type annotation and validation rules stay together.
Query validation	<pre>from fastapi import Query q: str = Query(min_length=3)</pre>	Applies constraints like <code>min_length</code> and <code>max_length</code> to query parameters.
Pattern validation	<pre>from typing import Annotated from fastapi import Query q: Annotated[str None, Query(pattern="^fixedquery\$")] = None</pre>	Validates a string parameter against a regex pattern with <code>pattern</code> (the older <code>regex</code> keyword was deprecated in its favor).
Path validation	<pre>from fastapi import Path id: int = Path(ge=1, le=1000)</pre>	Validates path parameters with numeric constraints like <code>ge</code> (greater/equal), <code>le</code> (less/equal).
Numeric constraints	<pre>price: float = Query(gt=0, le=9999.99)</pre>	Applies gt (greater than), lt (less than), ge , le constraints to numbers.
Required query parameter	<pre>q: Annotated[str, Query(min_length=1)]</pre>	- Modern approach: no default = required when using <code>Annotated</code> - older approach: <code>q: str = Query(...)</code> with ellipsis.
Optional with None	<pre>q: str None = None</pre>	- Python 3.10+ union syntax for optional parameters - older versions use <code>Optional[str]</code>.
List query parameter	<pre>tags: list[str] = Query([])</pre>	- Accepts multiple values for same parameter (<code>?tags=a&tags=b</code>) - validated as list.
Alias parameter	<pre>item_query: str = Query(alias="item-query")</pre>	Maps hyphenated URL param to <code>snake_case</code> Python variable.

Technique	Example	Description
Title and description metadata	<pre>from typing import Annotated from fastapi import Query q: Annotated[str None, Query(title="Query string", description="Search term")] = None</pre>	Adds a human readable title and description that show up in the generated OpenAPI docs.
Deprecated parameter	<pre>old_param: str = Query(deprecated=True)</pre>	• Marks parameter as deprecated in OpenAPI docs • still functional but discouraged.
Exclude from OpenAPI	<pre>from typing import Annotated from fastapi import Query hidden_query: Annotated[str None, Query(include_in_schema=False)] = None</pre>	Hides a parameter from the generated OpenAPI schema and docs with <code>include_in_schema=False</code> .
AfterValidator custom validation	<pre>from pydantic import AfterValidator def check_valid_id(id: str): if not id.startswith(("isbn-", "imdb-")): raise ValueError("Invalid ID") return id id: Annotated[str None, AfterValidator(check_valid_id)] = None</pre>	Runs a custom function after normal validation, for checks the built-in constraints cannot express.

TABLE 5

Pydantic Models and Validation

Pydantic models are the heart of how FastAPI validates and serializes structured data, a `BaseModel` subclass defines the shape of a request or response, and FastAPI does the rest. This table covers the tools for going beyond simple fields: nesting, reuse across input/output shapes, per-field constraints and examples, custom validators, and the extra data types Pydantic understands. Note this is Pydantic v2, so `model_config` and `field_validator` replace the older patterns you may have seen.

Concept	Example	Description
BaseModel	<pre>class Item(BaseModel): name: str price: float</pre>	• Base class for all Pydantic models • provides automatic validation and serialization.
Optional fields	<pre>description: str None = None</pre>	• Field can be omitted or null • a default value makes it not required.
Field constraints	<pre>from pydantic import Field price: float = Field(gt=0)</pre>	• Adds validation rules directly to model fields • similar to Query/Path but for models.
Nested models and lists	<pre>class Image(BaseModel): url: str class Item(BaseModel): images: list[Image] = []</pre>	• Uses one model as a field's type, or wraps it in <code>list[Model]</code> • each item is validated and documented.
Field description	<pre>name: str = Field(description="Item name")</pre>	• Adds documentation to a field • appears in the generated OpenAPI schema.
Field example	<pre>price: float = Field(examples=[9.99, 19.99])</pre>	Provides example values for API documentation and testing.
Model example data	<pre>model_config = { "json_schema_extra": { "examples": [{"name": "Foo", "price": 35.4}] } }</pre>	Adds sample data for the whole model to the generated JSON Schema, shown in the docs UI.
Field validator	<pre>@field_validator('email') def check_email(cls, v):</pre>	• Custom validation logic for a specific field • runs after type validation.
Model validator	<pre>@model_validator(mode='after') def check_passwords(self):</pre>	• Validates the entire model after individual fields • useful for cross-field validation.

Concept	Example	Description
model_config	<pre>from pydantic import ConfigDict model_config = ConfigDict(str_strip_whitespace=True)</pre>	- Configures model behavior using Pydantic v2's <code>ConfigDict</code> - replaces the old inner <code>class Config</code> .
Computed field	<pre>@computed_field @property def total(self) -> float:</pre>	- Adds a calculated field to a model - included in serialization but not validation.
Extra models	<pre>class UserIn(BaseModel): password: str class UserOut(BaseModel): ... class UserInDB(BaseModel): hashed_password: str</pre>	- Separate models for input, output and storage - keeps secrets like passwords out of responses.
Model inheritance (reduce duplication)	<pre>class UserBase(BaseModel): username: str class UserIn(UserBase): password: str</pre>	- A base model shares fields with subclasses reduces duplication across the extra-models pattern.
Partial updates	<pre>update_data = item.model_dump(exclude_unset=True)</pre>	- Returns only the fields the client actually sent - used for PATCH so unset fields keep their stored value.
jsonable_encoder	<pre>from fastapi.encoders import jsonable_encoder data = jsonable_encoder(item)</pre>	Converts a Pydantic model, or other object, into JSON-compatible types like <code>dict</code> and <code>str</code> .
Extra data types	<pre>item_id: UUID start_datetime: datetime</pre>	- Pydantic also validates <code>UUID</code> , <code>datetime</code> , <code>Decimal</code> and more - each serializes to its natural JSON form.
Dataclasses	<pre>from dataclasses import dataclass @dataclass class Item: name: str</pre>	- FastAPI accepts standard dataclasses instead of <code>BaseModel</code> for requests and responses - converted to Pydantic's own flavor internally.

TABLE 6

Response Configuration

Controlling what goes back to the client is just as important as parsing what comes in. The `response_model` and return-type annotation filter and validate your output, handy for stripping a password field before it ever leaves the server, while the various response classes let you return HTML, files, or redirects instead of plain JSON. Status codes, custom headers, and cookies round out the toolkit for shaping a precise response.

Technique	Example	Description
Response model	<pre>@app.post("/items", response_model=Item)</pre>	- Specifies return type for validation and docs - filters response to match model - since 0.130.0, Pydantic serializes it in Rust, over 2x faster than the old JSON encoder.
Return type annotation	<pre>async def create_item(item: Item) -> Item:</pre>	- Modern preferred approach for response type declaration - FastAPI uses the return annotation for validation, filtering, and docs.
Status code	<pre>@app.post("/items", status_code=201)</pre>	- Sets HTTP status code for the response - FastAPI's default is 200 for every method, 201 for POST is only a convention you set explicitly.
Response model exclude	<pre>response_model=User, response_model_exclude={"password"}</pre>	Removes specified fields from the serialized response only - the field is still fetched and held in memory first, so pair it with a separate output model for real secrets.
Response model exclude unset	<pre>response_model=Item, response_model_exclude_unset=True</pre>	- Returns only the fields explicitly set, dropping defaults - useful for models with many optional fields so responses stay short.

Technique	Example	Description
Response description	<pre>@app.post("/items/", response_description="The created item")</pre>	Sets the docs description for the response itself , separate from the path operation's own description.
JSONResponse	<pre>from fastapi.responses import JSONResponse return JSONResponse(content=data)</pre>	Explicit JSON response with custom headers or status code.
HTMLResponse	<pre>from fastapi.responses import HTMLResponse return HTMLResponse(content=html)</pre>	- Returns HTML content - sets Content-Type to text/html.
RedirectResponse	<pre>from fastapi.responses import RedirectResponse return RedirectResponse(url="/new")</pre>	HTTP redirect to different URL - status_code defaults to 307.
FileResponse	<pre>from fastapi.responses import FileResponse return FileResponse(path="file.pdf")</pre>	- Serves static files - handles content-type and download headers automatically.
Response headers	<pre>from fastapi import Response response.headers["X-Custom"] = "value"</pre>	Sets custom HTTP headers in response.
Set cookies	<pre>response.set_cookie(key="session", value="abc")</pre>	- Adds cookie to response - supports httponly, secure, samesite options.
Return a Response directly	<pre>from fastapi import Response return Response(content=xml_data, media_type="application/xml")</pre>	- Bypasses <code>response_model</code> conversion entirely, useful for non-JSON formats - the OpenAPI docs won't auto-document its media type.
Response parameter for status code	<pre>def get_or_create_task(task_id: str, response: Response): response.status_code = status.HTTP_201_CREATED</pre>	Sets the status code from inside the function body while still returning a plain object filtered by <code>response_model</code> .
Additional status codes	<pre>return JSONResponse(status_code=status. HTTP_201_CREATED, content=item)</pre>	Returns a second status code from the same path operation, alongside the default one declared in the decorator.
Default response class	<pre>app = FastAPI(default_response_class=H TMLResponse)</pre>	Sets the response class used by default for every path operation on an app or router.
Custom response class	<pre>class CustomORJSONResponse(Response): def render(self, content: Any) -> bytes: return orjson.dumps(content)</pre>	Subclasses <code>Response</code> and implements <code>render()</code> to control exactly how content becomes bytes.
ORJSONResponse (legacy)	<pre>from fastapi.responses import ORJSONResponse return ORJSONResponse(content=data)</pre>	Deprecated: a <code>response_model</code> already gets the same Rust-based serialization speed without a custom response class.

TABLE 7

Streaming Responses

Sending a response body in pieces instead of all at once, plain chunks with `StreamingResponse`, one JSON object per line with JSON Lines, or push events with built-in Server-Sent Events. Two-way channels stay in WebSockets.

Concept	Example	Description
StreamingResponse	<pre>from fastapi.responses import StreamingResponse @app.get("/story/stream", response_class=StreamingResponse) async def stream_story() -> AsyncIterable[str]: for line in message.splitlines(): yield line</pre>	- Streams a response in chunks by declaring <code>response_class=StreamingResponse</code> and using <code>yield</code> - sends data as it is produced, without holding the whole body in memory.

Concept	Example	Description
Server-Sent Events	<pre>from fastapi.sse import EventSourceResponse @app.get("/items/stream", response_class=EventSourceResponse) async def stream_items() -> AsyncIterable[Item]: for item in items: yield item</pre>	- Built-in one-way push from server to client over plain HTTP (<code>fastapi.sse</code> , added in 0.135.0) - simpler than WebSockets for AI chat tokens, logs and live notifications.
Stream JSON Lines	<pre>from collections.abc import AsyncIterable @app.get("/items/stream") async def stream_items() -> AsyncIterable[Item]: for item in items: yield item</pre>	- Sends one JSON object per line (<code>application/jsonl</code>) as each item is produced - declared by <code>yield</code> -ing from the path operation instead of <code>return</code> (added in 0.134.0).
ServerSentEvent	<pre>from fastapi.sse import EventSourceResponse, ServerSentEvent async def stream_items() -> AsyncIterable[ServerSentEvent]: yield ServerSentEvent(data=item, event="item_update", id=str(i), retry=5000)</pre>	- Sets the SSE <code>data</code> , <code>event</code> , <code>id</code> , <code>retry</code> or <code>comment</code> fields instead of yielding plain data <code>raw_data</code> sends pre-formatted text instead of JSON-encoding data .
Stream binary data with yield	<pre>class PNGStreamingResponse(StreamingResponse): media_type = "image/png" @app.get("/image/stream", response_class=PNGStreamingResponse) async def stream_image() -> AsyncIterable[bytes]: with read_image() as f: for chunk in f: yield chunk</pre>	- Streams bytes (images, audio) in chunks; subclassing <code>StreamingResponse</code> sets <code>media_type</code> for the right <code>Content-Type</code> <code>yield from</code> streams a file-like object without writing the loop.
Client disconnect in a stream	<pre>from starlette.requests import Request async def stream_items(request: Request): for item in items: if await request.is_disconnected(): break yield item</pre>	Checks mid-stream whether the client has dropped the connection, so the generator can stop producing more chunks.
Status code on a streaming endpoint	<pre>@app.get("/items/stream", status_code=201, response_class=EventSourceResponse,)</pre>	FastAPI 0.140.13 fixed <code>status_code</code> being ignored on SSE and JSON Lines streaming endpoints, the decorator's status code now applies.

TABLE 8

Dependency Injection

Dependency injection is one of FastAPI's most useful features: `Depends()` lets you factor out reusable logic like database sessions, authentication, and pagination, then declare it as a parameter wherever you need it. Dependencies can nest, run as `yield` generators that clean up after the response, apply at the route, router, or app level, and be swapped out entirely for testing. Lean on this pattern and your endpoints stay thin while shared concerns live in one place.

Pattern	Example	Description
Basic dependency	<pre>from typing import Annotated async def common_parameters(q: str None = None): return {"q": q} def route(common: Annotated[dict, Depends(common_parameters)]):</pre>	- Function providing reusable logic - recommended Annotated syntax (since 0.95.0) keeps type and dependency together.
Annotated type alias dependency	<pre>CommonsDep = Annotated[dict, Depends(common_parameters)] def endpoint(common: CommonsDep):</pre>	- Defines reusable type alias for common dependency - reduces repetition across many routes.

Pattern	Example	Description
Class dependency	<pre>class Pagination: def __init__(self, skip=0):</pre>	- Class instances as dependencies <code>__init__</code> parameters become sub-dependencies, validated the same way as path operation parameters, and give the editor type completion a dict cannot.
Nested dependencies	<pre>def query_extractor(q: str None = None): return q def query_or_cookie_extractor(q: Annotated[str, Depends(query_extractor)]):</pre>	- Dependency can depend on other dependencies - creates dependency chain; a sub-dependency shared by several dependants in one request still runs only once per request.
Yield dependency	<pre>def get_db(): db = Database() yield db db.close()</pre>	- Provides resource cleanup after the response is already sent, not before - if the code after <code>yield</code> catches an exception without re-raising it, FastAPI never learns the exception happened.
Async dependency	<pre>async def get_session(): async with Session() as s: yield s</pre>	Async generator dependency <code>sync (def)</code> and <code>async (async def)</code> dependencies can be freely mixed in the same app, FastAPI runs each correctly.
Global dependencies	<pre>app = FastAPI(dependencies=[Depends(verify)])</pre>	- Applied to all routes in application - useful for auth, logging; combines with router level and route level dependencies rather than overriding them.
Router dependencies	<pre>router = APIRouter(dependencies=[Depends(check)])</pre>	- Applied to all routes in router - scoped to specific module; when a route also has its own dependencies, FastAPI resolves the router's first, then the path decorator's, then normal parameter dependencies.
Dependencies in path decorator	<pre>@app.get("/items", dependencies=[Depends(verify)])</pre>	- Dependency executed but result not used in function - runs unconditionally for its side effects only, such as raising an error on an invalid header.
Dependency module import	<pre>from fastapi import APIRouter, Depends from ..dependencies import get_token_header router = APIRouter(dependencies=[Depends (get_token_header)])</pre>	- Import a dependency function defined in one shared module (e.g. <code>app/dependencies.py</code>) into any router package - keeps one definition reused across the app instead of duplicated per router.
Parameterized dependencies	<pre>class FixedContentQueryChecker: def __init__(self, fixed_content: str): self.fixed_content = fixed_content def __call__(self, q: str = ""): return self.fixed_content in q checker = FixedContentQueryChecker("bar")</pre>	- A callable class instance (<code>__call__</code>) used as a dependency - lets <code>__init__</code> parameterize the same dependency logic instead of writing many near-identical functions.
Security() dependency	<pre>current_user: Annotated[User, Security(get_current_user, scopes=["me"])]</pre>	<code>Security()</code> is <code>Depends()</code> plus an OAuth2 scopes list, declared where the dependency is used - each route or sub-dependency can require a different scope from the same security dependency.
use_cache=False	<pre>async def needy_dependency(fresh_value: Annotated[str, Depends(get_value, use_cache=False)]):</pre>	- A dependency reused several times in one request is called once and cached by default <code>use_cache=False</code> forces FastAPI to call it again each time it is declared.
Response parameter in dependencies	<pre>def set_status(response: Response): response.status_code = 201</pre>	- Declare <code>response: Response</code> as a parameter inside a dependency, not only in the path operation function - lets a dependency set the status code, headers or cookies; if several set it, the last one wins.

TABLE 9

Error Handling

When something goes wrong, you want a clean, consistent error response rather than a raw stack trace. Raising `HTTPException` is the everyday move for returning a 404 or 403, while custom exceptions paired with exception handlers let you centralize how whole categories of errors turn into responses. You can even override FastAPI's default validation-error format when you need errors shaped a particular way.

Approach	Example	Description
HTTPException	<pre>from fastapi import HTTPException raise HTTPException(status_code=404, detail="Not found")</pre>	• Built-in exception for HTTP errors • caught by a default handler and formatted as JSON automatically, no setup needed.
Status code in exception	<pre>raise HTTPException(status_code=403, detail="Forbidden")</pre>	• Sets the HTTP status code directly when raising the exception • common codes: 400, 401, 403, 404, 500.
JSON-able exception detail	<pre>raise HTTPException(status_code=404, detail={"msg": "Item not found", "id": item_id})</pre>	• <code>detail</code> accepts any JSON-able value, not just a string • pass a dict or list and it is converted to JSON automatically
Custom headers in exception	<pre>raise HTTPException(status_code=404, detail="Item not found", headers={"X-Error": "There goes my error"},)</pre>	Adds custom headers to an error response, useful for auth challenges and other advanced scenarios.
Custom exception	<pre>class CustomError(Exception): pass</pre>	• Define a custom exception class by subclassing plain <code>Exception</code> • does nothing on its own until paired with a handler.
Exception handler	<pre>@app.exception_handler(CustomError) def handler(request, exc):</pre>	• Catches a specific exception type globally, and any subclass of it too • returns a custom response.
Validation error handler	<pre>from fastapi.exceptions import RequestValidationError @app.exception_handler(RequestValidationE rror)</pre>	• Customizes Pydantic's 422 validation error response format • needs its own registration, overriding <code>HTTPException</code>'s handler alone does not affect it.
RequestValidationError or body	<pre>async def handler(request, exc: RequestValidationError): return JSONResponse(status_code=422, content={"detail": exc.errors(), "body": exc.body})</pre>	<code>.errors()</code> lists the validation failures and <code>.body</code> holds the invalid request body received , useful for logging or returning to the caller.
Override default handlers	<pre>from starlette.exceptions import HTTPException as StarletteHTTPException @app.exception_handler(StarletteHTTPExcep tion)</pre>	• Replaces FastAPI's default error formatting • registers on Starlette's <code>HTTPException</code> so it also catches errors Starlette raises internally, not just FastAPI's own subclass.
Reuse default exception handlers	<pre>from fastapi.exception_handlers import (http_exception_handler, request_validation_exception_handler) return await http_exception_handler(request, exc)</pre>	Imports FastAPI's own default handlers from <code>fastapi.exception_handlers</code> so a custom handler can log or tweak, then fall back to the default behavior .

TABLE 10

Authentication and Security

FastAPI ships with security utilities that plug straight into its dependency system and its OpenAPI docs. The rows below walk through the common building blocks: OAuth2 password flows, API keys sent via headers, query parameters, or cookies, JWT creation and verification with PyJWT, and password hashing with pwdlib, and how they combine into a `get_current_user` dependency you reuse across protected routes. Security scopes at the end add fine-grained, permission-based access on top.

Mechanism	Example	Description
OAuth2 password flow	<pre>from fastapi.security import OAuth2PasswordBearer oauth2 = OAuth2PasswordBearer(tokenUrl="token")</pre>	• Standard OAuth2 flow for password-based authentication • extracts the bearer token string from the header only, your own dependency must still verify it.
OAuth2 password request	<pre>from fastapi.security import OAuth2PasswordRequestForm def login(form: OAuth2PasswordRequestForm = Depends()):</pre>	• Form data model for username/password login • must be sent as form fields, not JSON, the field names are fixed by the OAuth2 spec.
Current user dependency	<pre>async def get_current_user(token = Depends(oauth2)): # decode token, look up user, check active</pre>	• Dependency that decodes the token and looks up the user • also checks the account is still active, a valid signature alone is not enough.
JWT token creation (encode)	<pre>import jwt token = jwt.encode(data, SECRET, algorithm="HS256")</pre>	• Encodes a JWT token with a payload and a secret key • FastAPI's current docs use the actively maintained PyJWT library (<code>import jwt</code>), not the older python-jose.
JWT token verification (decode)	<pre>import jwt from jwt.exceptions import InvalidTokenError payload = jwt.decode(token, SECRET, algorithms=["HS256"])</pre>	• Decodes and verifies a JWT's signature • raises <code>InvalidTokenError</code> if invalid, the payload itself is only base64-encoded, never encrypted.
Password hashing	<pre>from pwdlib import PasswordHash password_hash = PasswordHash.recommended()</pre>	• Hashes passwords for storage, one-way, never store plain passwords • FastAPI's current docs recommend pwdlib with Argon2, bcrypt remains supported for compatibility with older hashes.
Dummy hash against timing attacks	<pre>def authenticate_user(fake_db, username, password): user = get_user(fake_db, username) if not user: verify_password(password, DUMMY_HASH) return False</pre>	• Runs <code>verify_password</code> against a dummy hash even when the username doesn't exist • keeps response time constant, closing a timing side-channel that could reveal valid usernames.
HTTPBearer scheme	<pre>from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials security = HTTPBearer()</pre>	• Extracts a raw bearer token from the <code>Authorization</code> header as an <code>HTTPAuthorizationCredentials</code> object • unlike <code>OAuth2PasswordBearer</code>, it needs no token URL, useful for pre-issued or third-party tokens.
API key in header	<pre>from fastapi.security import APIKeyHeader api_key = APIKeyHeader(name="X-API-Key")</pre>	• Extracts API key from a custom header • only reads the value, your code must still check that it is a valid key.
API key in query	<pre>from fastapi.security import APIKeyQuery api_key = APIKeyQuery(name="api_key")</pre>	• Reads API key from a query parameter • less secure than a header, query strings often end up in server logs and browser history.
API key in cookie	<pre>from fastapi.security import APIKeyCookie api_key = APIKeyCookie(name="session")</pre>	• Extracts API key from a cookie • cookies are attached to requests automatically by the browser, which is what makes cookie-based auth CSRF-prone without extra defenses.

Mechanism	Example	Description
HTTP Basic auth	<pre>from fastapi.security import HTTPBasic, HTTPBasicCredentials security = HTTPBasic() is_ok = secrets.compare_digest(credentials.username.encode("utf8"), b"stanleyjobson")</pre>	• Browser-native username/password prompt, no login form needed • check credentials with <code>secrets.compare_digest</code>, a plain <code>==</code> leaks timing information.
Optional authentication	<pre>api_key = APIKeyHeader(name="X-API-Key", auto_error=False)</pre>	• <code>auto_error=False</code> makes the dependency return <code>None</code> instead of erroring when the credential is missing • useful for routes with optional or multiple possible auth methods.
Security scopes	<pre>async def read_own_items(current_user: Annotated[User, Security(get_current_active_user, scopes=["items"])],):</pre>	• Permission-based access control, declared per route with <code>Security()</code> instead of <code>Depends()</code> • the dependency receives a <code>SecurityScopes</code> object and must compare the token's scopes against what is required.

TABLE 11

Middleware

Middleware wraps every request and response, making it the right place for cross-cutting concerns that shouldn't live in individual endpoints. CORS is the one nearly everyone needs first, while the others handle compression, HTTPS enforcement, trusted-host checks, and sessions, plus the option to write your own function- or class-based middleware, or drop to the pure ASGI layer, for custom logic like timing or request logging.

Type	Example	Description
CORS middleware	<pre>from fastapi.middleware.cors import CORSMiddleware app.add_middleware(CORSMiddleware, allow_origins=["https://example.com"], allow_credentials=True)</pre>	• Enables Cross-Origin Resource Sharing • configures which domains can access API
Custom function middleware	<pre>@app.middleware("http") async def add_process_time_header(request, call_next): response = await call_next(request) response.headers["X-Process-Time"] = str(time.perf_counter()) return response</pre>	• Function-based middleware • runs before and after every request
Custom class middleware	<pre>class CustomHeaderMiddleware(BaseHTTPMiddleware): async def dispatch(self, request, call_next): response = await call_next(request) response.headers["Custom"] = "Example" return response</pre>	• Class-based middleware via <code>BaseHTTPMiddleware</code> • supports configuration via <code>__init__</code> for more complex logic
Middleware execution order	<pre>app.add_middleware(MiddlewareA) app.add_middleware(MiddlewareB) # Request: MiddlewareB -> MiddlewareA -> route # Response: route -> MiddlewareA -> MiddlewareB</pre>	• Middleware forms a stack: the last one added is the outermost layer • runs first on the request path, last on the response path
GZip middleware	<pre>from fastapi.middleware.gzip import GZipMiddleware app.add_middleware(GZipMiddleware, minimum_size=1000)</pre>	• Compresses responses over specified size • reduces bandwidth
Trusted host middleware	<pre>from fastapi.middleware.trustedhost import TrustedHostMiddleware app.add_middleware(TrustedHostMiddleware, allowed_hosts=["*.example.com"])</pre>	• Validates Host header • prevents HTTP Host header attacks
HTTPS redirect middleware	<pre>from fastapi.middleware.httpsredirect import HTTPSRedirectMiddleware app.add_middleware(HTTPSRedirectMiddleware)</pre>	• Redirects HTTP to HTTPS • enforces secure connections

Type	Example	Description
Session middleware	<pre>from starlette.middleware.sessions import SessionMiddleware app.add_middleware(SessionMiddleware, secret_key="key")</pre>	Adds session support via signed cookies
Pure ASGI middleware class	<pre>class ASGIMiddleware: def __init__(self, app): self.app = app async def __call__(self, scope, receive, send): await self.app(scope, receive, send)</pre>	• Implements the ASGI interface directly (<code>scope</code> , <code>receive</code> , <code>send</code>) instead of <code>BaseHTTPMiddleware</code> • avoids <code>BaseHTTPMiddleware</code> 's contextvar propagation limits

TABLE 12

Background Tasks

Some work, like sending a confirmation email or writing a log line, shouldn't make the user wait for the response. FastAPI's built-in `BackgroundTasks` handles these lightweight cases right inside the same process, with no extra infrastructure required. For heavier or long-running jobs that need their own workers, this table also covers Celery and ARQ as the next step up.

Approach	Example	Description
BackgroundTasks	<pre>from fastapi import BackgroundTasks def task(email: str): send_email(email)</pre>	• Simple background function executed after response • no external queue needed.
Add background task	<pre>def endpoint(bg: BackgroundTasks): bg.add_task(send_email, to="user@example.com")</pre>	• Schedules function to run after response sent • tasks run in same process.
Multiple background tasks	<pre>bg.add_task(task1) bg.add_task(task2)</pre>	• Multiple tasks run sequentially in order added • an exception in one stops the rest from running
Background task with dependencies	<pre>def get_query(background_tasks: BackgroundTasks, q: str = None): background_tasks.add_task(write_log, message)</pre>	• A dependency can receive <code>BackgroundTasks</code> and call <code>.add_task()</code> itself • its data is resolved during the request, before the task runs, not by <code>Depends()</code> on the task function itself.
Celery integration	<pre>from celery import Celery celery = Celery("tasks", broker="redis://localhost") @celery.task def heavy_task():</pre>	• Distributed task queue for heavy/long-running jobs • runs in separate workers outside request cycle.
ARQ for async tasks	<pre>from arq import create_pool redis = await create_pool()</pre>	• Redis-based async task queue built on <code>asyncio</code> • alternative to Celery for async-first applications.

TABLE 13

Database Integration

FastAPI's own tutorial uses `SQLModel`, a thin layer over `SQLAlchemy` and `Pydantic`, but plain `SQLAlchemy` 2.0 sessions and models are just as common in production. These rows cover defining models either way, injecting a session as a dependency so it closes cleanly after each request, and running queries; `Alembic` handles schema changes and pooling keeps a busy API from exhausting the database.

Technique	Example	Description
SQLAlchemy async session	<pre>from sqlalchemy.ext.asyncio import AsyncSession async def get_db(): yield session</pre>	• Async database session for non-blocking DB operations • requires an <code>asyncio</code>-native driver like <code>asyncpg</code> or <code>aiosqlite</code>.
Database dependency	<pre>def endpoint(db: Session = Depends(get_db)):</pre>	• Injects database session into route • session closed automatically after request via the dependency's <code>yield</code> / <code>finally</code> .

Technique	Example	Description
SessionDep	<pre>def get_session(): with Session(engine) as session: yield session SessionDep = Annotated[Session, Depends(get_session)]</pre>	• Annotated type alias for the session dependency • reused as a parameter type on every route instead of repeating <code>Depends(get_session)</code> .
SQLModel	<pre>from sqlmodel import Field, SQLModel class Hero(SQLModel, table=True): id: int None = Field(default=None, primary_key=True) name: str = Field(index=True)</pre>	• ORM used by FastAPI's own SQL tutorial • a model class with <code>table=True</code> maps directly to a database table.
Separate table and data models	<pre>class HeroBase(SQLModel): name: str class Hero(HeroBase, table=True): id: int None = Field(default=None, primary_key=True) class HeroPublic(HeroBase): id: int class HeroCreate(HeroBase): secret_name: str</pre>	• Splits the table model from the input/output models (<code>HeroCreate</code> , <code>HeroPublic</code>) via a shared base • keeps secret or internal fields out of API responses.
Database model	<pre>from sqlalchemy.orm import Mapped, mapped_column class User(Base): id: Mapped[int] = mapped_column(primary_key=True)</pre>	• ORM model defining table structure via typed columns • the modern <code>Mapped / mapped_column()</code> style replaces the older <code>Column(...)</code> form.
Async database query	<pre>result = await session.execute(select(User)) users = result.scalars().all()</pre>	• Executes async query • uses SQLAlchemy 2.0 style.
SQLAlchemy Session	<pre>from sqlalchemy.orm import Session with Session(engine) as session: session.add(obj) session.commit()</pre>	• The synchronous counterpart to <code>AsyncSession</code> , blocks the thread during each query • opened as a context manager and closed automatically on exit.
Alembic migrations	<pre>alembic init alembic alembic revision --autogenerate alembic upgrade head</pre>	• Database schema migrations • tracks and applies schema changes over time; always review autogenerated migrations before applying, since they can miss renames.
Connection pooling	<pre>engine = create_async_engine(url, pool_size=20, max_overflow=10)</pre>	• Reuses database connections instead of opening a new one per query • critical for production performance.

TABLE 14

WebSockets

When you need a live, two-way channel, like chat, notifications, or a streaming dashboard, a plain request/response cycle won't do. FastAPI's WebSocket support gives you the full lifecycle: accept the connection, receive and send text or JSON in a loop, and handle the disconnect cleanly. The final SSE row is the simpler choice when updates only need to flow one way, from server to client.

Feature	Example	Description
WebSocket endpoint	<pre>from fastapi import WebSocket @app.websocket("/ws") async def websocket_endpoint(websocket: WebSocket):</pre>	• Defines WebSocket route • full-duplex communication channel.
Accept connection	<pre>await websocket.accept()</pre>	• Establishes WebSocket connection • must be called before send/receive.
Receive message	<pre>data = await websocket.receive_text() data = await websocket.receive_json()</pre>	• Reads message from client • supports text, JSON, or bytes.

Feature	Example	Description
Send message	<pre>await websocket.send_text("Hello") await websocket.send_json({"msg": "Hi"})</pre>	Sends message to client.
WebSocket disconnect	<pre>try: while True: await websocket.receive_text() except WebSocketDisconnect:</pre>	- Handles client disconnect - exception raised when connection closes.
Dependencies in WebSocket endpoint	<pre>@app.websocket("/items/{item_id}/ws") async def websocket_endpoint(websocket: WebSocket, item_id: str, q: Optional[int] = None, cookie_or_token: Annotated[str, Depends(get_cookie_or_token)],):</pre>	- Path and query parameters, plus Depends , Cookie , Header and Security , work in a WebSocket endpoint just like in HTTP routes - the item ID comes from the path, q from the query string
Connection manager (broadcast)	<pre>class ConnectionManager: def __init__(self): self.active_connections: list[WebSocket] = [] async def broadcast(self, message: str): for c in self.active_connections: await c.send_text(message)</pre>	- Keeps every open WebSocket in a list to message all connected clients - in-memory only, works within a single running process
Close with a code (WebSocketException)	<pre>from fastapi import WebSocketException, status raise WebSocketException(code=status.WS_1008_ POLICY_VIOLATION)</pre>	Closes a WebSocket from an endpoint or dependency with a spec-defined close code - use this instead of HTTPException , which does not apply to WebSockets

TABLE 15

Testing

FastAPI makes testing genuinely pleasant because **TestClient** runs your app in-process, no server to spin up. These rows show the standard pytest workflow, the context-manager form that triggers lifespan events, the httpx-based async client for testing async code properly, and the dependency-override trick that swaps a real dependency for a mock without touching application code.

Tool	Example	Description
TestClient	<pre>from fastapi.testclient import TestClient client = TestClient(app) response = client.get("/")</pre>	Synchronous test client - tests FastAPI without running server.
Test with pytest	<pre>def test_read_main(): response = client.get("/") assert response.status_code == 200</pre>	- Standard pytest patterns work with TestClient - uses fixtures for setup.
TestClient as context manager	<pre>with TestClient(app) as client: response = client.get("/")</pre>	Triggers lifespan events during testing - startup/shutdown run automatically.
Async test client	<pre>@pytest.mark.anyio async def test_root(): async with AsyncClient(transport=ASGITransp ort(app=app), base_url="http://test") as ac: response = await ac.get("/")</pre>	Async testing with httpx, marked with @pytest.mark.anyio so pytest calls the test function asynchronously - lets a test await other async code, like database calls.
Override dependency	<pre>async def override_dependency(q: Optional[str] = None): return {"q": q, "skip": 5, "limit": 10} app.dependency_overrides[common_parameters] = override_dependency</pre>	Replaces a dependency's return value for tests, e.g. swapping a slow external call for a fixed one - no application code changes needed.
Test file upload	<pre>files = {"file": ("test.txt", b"content", "text/plain")} client.post("/upload", files=files)</pre>	Simulates file upload in tests, same files= argument httpx uses - tuple format: (filename, content, content-type).

Tool	Example	Description
Test WebSocket	<pre>with client.websocket_connect("/ws") as websocket: data = websocket.receive_json() assert data == {"msg": "Hello WebSocket"}</pre>	Uses the same <code>TestClient</code> , in a <code>with</code> block, to connect to a WebSocket route and exchange messages.
Test with pytest fixtures	<pre>@pytest.fixture def client(): return TestClient(app) def test_read_main(client): assert client.get("/").status_code == 200</pre>	• Reusable test setup: pytest runs the fixture and passes its return value in • requested by name as a test function parameter.
raise_server_exceptions	<pre>client = TestClient(app, raise_server_exceptions=False)</pre>	• By default <code>TestClient</code> re-raises unhandled server exceptions instead of returning a 500 response • set <code>False</code> to assert on the error response instead.

TABLE 16

Advanced Routing

As an app grows past a single file, `APIRouter` is how you split routes into focused modules with shared prefixes, tags, and dependencies, then mount them onto the main app with `include_router()`. Sub-application mounting, WSGI mounting and static file serving are a different tool entirely, since they hand a path over to a genuinely separate app instead of just organizing routes. The route-priority row captures a subtle gotcha: declare `/users/me` before `/users/{id}`, or the parameterized route swallows it silently.

Concept	Example	Description
APIRouter	<pre>from fastapi import APIRouter router = APIRouter(prefix="/items", tags=["items"])</pre>	• Modular routing • groups related endpoints with shared prefix and tags.
Include router	<pre>app.include_router(users.router) app.include_router(admin.router, prefix="/admin", tags=["admin"], dependencies=[Depends(get_token_header)])</pre>	• Mounts a router onto the main app • prefix, tags, dependencies and responses passed here extend the router's own without modifying it.
Router dependencies	<pre>router = APIRouter(dependencies=[Depends(get_token_header)])</pre>	• Applies dependency to all routes in the router • runs before per-route and per-decorator dependencies.
Router tags	<pre>router = APIRouter(tags=["items"])</pre>	• Groups a router's endpoints in the OpenAPI docs • creates collapsible sections in the interactive UI.
Order matters (route priority)	Define more specific routes first : <pre>@app.get("/users/me") @app.get("/users/{id}")</pre>	• FastAPI matches first matching route • specific paths before parameterized ones.
Static files	<pre>from fastapi.staticfiles import StaticFiles app.mount("/static", StaticFiles(directory="static"))</pre>	Serves static files from a directory.
Sub-application mounting	<pre>subapi = FastAPI() app.mount("/subapi", subapi)</pre>	• Mounts an independent FastAPI app at a path • that sub-app gets its own OpenAPI schema and docs UI.
WSGIMiddleware	<pre>from a2wsgi import WSGIMiddleware app.mount("/v1", WSGIMiddleware(flask_app))</pre>	• Wraps a WSGI app (Flask, Django) so it can be mounted under a path • requires the <code>a2wsgi</code> package.
Custom APIRoute class	<pre>class TimedRoute(APIRoute): def get_route_handler(self): ... router = APIRouter(route_class=TimedRoute)</pre>	• Subclasses APIRoute to customize request handling for a whole router • set via <code>route_class</code> on an <code>APIRouter</code> or <code>app.router.route_class</code>.

TABLE 17

OpenAPI Customization

FastAPI generates an OpenAPI schema and interactive docs for free, but real projects want to shape them. These rows let you document each route, add example responses, control the docs UIs, and extend the schema with callbacks, webhooks and client-generation hints.

Element	Example	Description
Metadata	<pre>app = FastAPI(title="My API", description="Docs", version="1.0")</pre>	Sets API title, description, version in OpenAPI schema and docs.
Tags metadata	<pre>tags_metadata = [{"name": "items", "description": "Item operations"}] app = FastAPI(openapi_tags=tags_metadata)</pre>	Adds descriptions to tag groups in documentation.
Summary and description	<pre>@app.post("/items/", summary="Create an item", description="Create an item with all the information",)</pre>	Sets a per-route summary and description shown in the interactive docs.
Description from docstring	<pre>@app.post("/items/", summary="Create an item") async def create_item(item: Item) -> Item: """Create an item with all the information"""</pre>	Reads the operation's description from its docstring (Markdown supported) instead of a <code>description=</code> argument.
Deprecate a path operation	<pre>@app.get("/old", deprecated=True)</pre>	• Marks endpoint as deprecated in docs • still functional.
Additional responses	<pre>@app.get("/items/{item_id}", response_model=Item, responses={404: {"model": Message}})</pre>	• Documents extra status codes and models for one route via <code>responses={...}</code> • can also add example payloads or extra media types per status.
Operation ID	<pre>@app.get("/items/", operation_id="some_specific_id_you_define")</pre>	• Sets unique operation identifier in OpenAPI • useful for code generation tools.
Disable docs	<pre>app = FastAPI(docs_url=None, redoc_url=None)</pre>	• Hides the Swagger UI and ReDoc pages • the <code>/openapi.json</code> schema stays public by default, set <code>openapi_url=None</code> too if it needs to be gone.
Custom docs URL	<pre>app = FastAPI(docs_url="/api/docs")</pre>	Changes Swagger UI path from default <code>/docs</code> .
Configure Swagger UI	<pre>app = FastAPI(swagger_ui_parameters={"syntaxHighlight": False})</pre>	Customizes Swagger UI behavior like syntax highlighting, theme, model expansion.
Custom OpenAPI schema	<pre>def custom_openapi(): # modify schema app.openapi = custom_openapi</pre>	• Overrides OpenAPI generation • add custom extensions or modify schema • caches the result in <code>app.openapi_schema</code> so it is only built once.
openapi_extra	<pre>@app.get("/items/", openapi_extra={"x-aperture-labs-portal": "blue"})</pre>	Injects arbitrary raw fields into one path operation's OpenAPI schema , such as <code>x-</code> extensions.
Separate input/output schemas	<pre>app = FastAPI(separate_input_output_schemas=False)</pre>	• FastAPI 0.102.0+ generates separate OpenAPI schemas for input and output with Pydantic v2 • set <code>False</code> to disable.
Conditional OpenAPI by environment	<pre>class Settings(BaseSettings): openapi_url: str = "/openapi.json" app = FastAPI(openapi_url=Settings().openapi_url)</pre>	Builds <code>openapi_url</code> from a setting or env var , so an empty value disables OpenAPI and both docs UIs in production.
Self-hosting docs UI static assets	<pre>app = FastAPI(docs_url=None) app.mount("/static", StaticFiles(directory="static"), name="static")</pre>	Serves Swagger UI's own JS and CSS from your app instead of the default CDN, for offline or restricted networks .

Element	Example	Description
Generate SDK clients	<pre>def custom_generate_unique_id(route: APIRoute): return f"{route.tags[0]}-{route.name}" app = FastAPI(generate_unique_id_function=custom_generate_unique_id)</pre>	- Feeds the OpenAPI schema to a client generator to build a typed SDK <code>generate_unique_id_function</code> gives generated client methods cleaner names.
OpenAPI callbacks	<pre>invoices_callback_router = APIRouter() @invoices_callback_router.post("/{callback_url}/invoices/{request.body.id}") def invoice_notification(body: InvoiceEvent): ... @app.post("/invoices/", callbacks=invoices_callback_router.routes)</pre>	Documents an external API your own endpoint will call back into , shown in OpenAPI as the request your API is expected to send.
OpenAPI webhooks	<pre>@app.webhooks.post("new-subscription") def new_subscription(body: Subscription): ...</pre>	Documents requests your app sends to the API consumer (a webhook), listed in the OpenAPI schema and docs.

TABLE 18

Settings and Configuration

Pydantic Settings brings the same type-safe validation you use for requests to your application config, reading values from environment variables and `.env` files and failing fast at startup if something's missing. Injecting settings as a dependency keeps them easy to mock in tests, and caching them with `lru_cache` means the config is parsed once. This is Pydantic Settings v2, so `SettingsConfigDict` replaces the old inner `Config` class.

Technique	Example	Description
Pydantic Settings	<pre>from pydantic_settings import BaseSettings class Settings(BaseSettings): admin_email: str</pre>	Type-safe configuration from environment variables - validates config at startup.
Read .env file	<pre>from pydantic_settings import SettingsConfigDict model_config = SettingsConfigDict(env_file=".env")</pre>	- Loads environment variables from .env using Pydantic Settings v2 - replaces old inner <code>class Config: env_file = ".env"</code> pattern.
Settings dependency	<pre>def get_settings(): return Settings() settings: Settings = Depends(get_settings)</pre>	Injects settings into routes - enables easy mocking in tests.
Cached settings	<pre>@lru_cache def get_settings(): return Settings()</pre>	Loads settings once and caches - improves performance.
Multiple environments	<pre>class Settings(BaseSettings): model_config = SettingsConfigDict(env_file=('.env', '.env.prod'))</pre>	- Loads multiple .env files in order - a later file like <code>.env.prod</code> overrides values from <code>.env</code>.

TABLE 19

Performance and Deployment

FastAPI is already fast, but these levers squeeze out more under real load and get it running in production. `async def` unlocks concurrency for I/O-bound code, and any plain `def` FastAPI calls for you still runs safely in a threadpool. The deployment rows, from a single Uvicorn process to workers, Docker, a reverse proxy, HTTPS and FastAPI Cloud, cover taking that app live.

Strategy	Example	Description
Async operations	<pre>async def fetch_data(): return await http_client.get(url)</pre>	- Use <code>async def</code> for I/O-bound operations - enables concurrent request handling, but a CPU-heavy <code>async def</code> with no real <code>await</code> still blocks the whole event loop.

Strategy	Example	Description
Threadpool for blocking code	<pre>def read_items(): return blocking_client.get(url)</pre>	- A plain <code>def</code> path operation or dependency is run in an external threadpool instead of blocking the event loop - use it when a library has no <code>await</code> support, but <code>async def</code> is still faster when the code truly awaits.
Uvicorn workers	<pre>fastapi run --workers 4 main.py</pre>	- Runs multiple worker processes to use multiple CPU cores - each worker is a separate process with its own memory, so in-memory caches or rate limiters aren't shared across workers.
Docker deployment	<pre>FROM python:3.14 WORKDIR /code COPY ./requirements.txt /code/requirements.txt RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt COPY ./app /code/app CMD ["fastapi", "run", "app/main.py", "--port", "80"]</pre>	- Official Docker image pattern for FastAPI production deployment - on Kubernetes or similar clusters, run a single process per container and let the cluster handle replication instead of adding <code>--workers</code> too.
Running behind a proxy	<pre>fastapi run main.py --forwarded-allow-ips="*" --root-path /api/v1</pre>	<code>--forwarded-allow-ips</code> trusts a proxy's <code>X-Forwarded-*</code> headers for the real client IP, protocol and host <code>--root-path</code> tells FastAPI it is served under a path prefix the proxy strips.
Redis caching	<pre>FastAPIRedis(app).lifespan().caching() @app.get("/items/{id}", dependencies=[Depends(cache(ttl=60))])</pre>	Caches responses in Redis to reduce database load for repeated queries - needs an explicit eviction step, or users can see stale data until the TTL expires.
Pydantic Rust JSON serialization	<pre>@app.post("/items/") async def create_item(item: Item) -> Item: return item</pre>	- A response model or return type lets FastAPI serialize straight to JSON with Pydantic's Rust core, skipping <code>jsonable_encoder</code> - this beats a custom <code>ORJSONResponse</code>, so skip <code>response_class</code> for maximum performance.
HTTPS termination	Traefik, Caddy, Nginx, or HAProxy terminate TLS and forward plain HTTP to Uvicorn	- A TLS Termination Proxy decrypts HTTPS from the client and forwards plain HTTP to the app server - certificates usually come free and auto-renewed from Let's Encrypt.
FastAPI Cloud	<pre>fastapi deploy</pre>	- Deploys the app with one command, built by the FastAPI team itself - also handles HTTPS and autoscaling replication; any other cloud provider needs its own setup.
Gunicorn + Uvicorn (legacy)	<pre>gunicorn main:app --workers 4 --worker-class uvicorn.workers.UvicornWorker</pre>	- Older process manager combo that starts and restarts Uvicorn worker processes - superseded now that Uvicorn and the <code>fastapi</code> CLI support <code>--workers</code> directly; the official <code>tiangolo/uvicorn-gunicorn-fastapi</code> image built on it is deprecated.

TABLE 20

Advanced Patterns

These are the extras reached for once the basics are in place and the app is heading toward production: rate limiting against abuse, shared lifespan state for expensive resources like a database or HTTP client, and server-rendered HTML through Jinja2Templates. GraphQL, class-based views and MessagePack serialization cover more specialized needs, and request-ID tracking adds the traceability a production API needs for its logs.

Pattern	Example	Description
Rate limiting	<pre>from slowapi import Limiter @limiter.limit("5/minute") def endpoint():</pre>	Throttles requests per IP/user - prevents API abuse

Pattern	Example	Description
Lifespan state shared with requests	<pre>async def lifespan(app): async with httpx.AsyncClient() as client: yield {"http_client": client} async def endpoint(request: Request): client = request.state.http_client</pre>	• Shares objects created in lifespan (a DB pool, an HTTP client) with every request via <code>request.state</code> • avoids global variables
Jinja2Templates	<pre>from fastapi.templating import Jinja2Templates templates = Jinja2Templates(directory="templates") @app.get("/items/{id}", response_class=HTMLResponse) async def read_item(request: Request, id: str): return templates.TemplateResponse(request=request, name="item.html", context={"id": id})</pre>	• Renders server-side HTML templates with Jinja2 • pass data through the <code>context</code> dictionary
Request ID tracking	<pre>from fastapi_request_context import RequestContextMiddleware app = RequestContextMiddleware(app)</pre>	• Generates a unique request ID per request and adds it to responses • ties log lines together for distributed tracing
GraphQL integration	<pre>from strawberry.fastapi import GraphQLRouter graphql_app = GraphQLRouter(schema) app.include_router(graphql_app, prefix="/graphql")</pre>	• Adds a GraphQL endpoint using Strawberry, FastAPI's own recommended library • enables a flexible query language
Class-based views	<pre>from fastapi_utils.cbv import cbv router = APIRouter() @cbv(router) class ItemCBV: session: Session = Depends(get_db) @router.get("/item/{item_id}")</pre>	• Groups related endpoints in a class • shares dependencies through <code>self</code> instead of repeating them
MessagePack serialization	<pre>from msgpack_asgi import MessagePackMiddleware app.add_middleware(MessagePackMiddleware)</pre>	• Uses a binary format for smaller payloads • content-negotiated via the <code>Accept</code> header

References

Official Documentation

1. Install FastAPI (fastapi[standard])
2. FastAPI app instance
3. Run with FastAPI CLI
4. Run with Uvicorn
5. Debugging with uvicorn.run
6. Async path operation
7. Lifespan context manager
8. Pydantic integration
9. app.frontend()
10. POST request
11. PUT request
12. DELETE request
13. Path parameter
14. Query parameter
15. Query parameter model
16. Multiple body parameters
17. Header parameter
18. Cookie parameter
19. Header/Cookie parameter model
20. Form data
21. Form model
22. File upload
23. Using the Request object directly
24. strict_content_type
25. Annotated query validation
26. Path validation
27. BaseModel
28. Optional fields
29. Nested models and lists
30. Model example data
31. Field validator
32. model_config
33. Extra models
34. jsonable_encoder
35. Extra data types
36. Dataclasses
37. Response model
38. Status code
39. Response description
40. JSONResponse
41. Response headers
42. Set cookies
43. Return a Response directly
44. Response parameter for status code
45. Additional status codes
46. StreamingResponse
47. Server-Sent Events
48. Stream JSON Lines
49. Status code on a streaming endpoint
50. Basic dependency
51. Class dependency
52. Nested dependencies

- 53. Yield dependency
- 55. Router dependencies
- 57. Parameterized dependencies
- 59. HTTPException
- 61. OAuth2 password request
- 63. HTTP Basic auth
- 65. CORS middleware
- 67. GZip middleware
- 69. Celery integration
- 71. SessionDep
- 73. SQLAlchemy Session
- 75. WebSocket endpoint
- 77. TestClient as context manager
- 79. Override dependency
- 81. Test with pytest fixtures
- 83. Sub-application mounting
- 85. Custom APIRoute class
- 87. Additional responses
- 89. Configure Swagger UI
- 91. Separate input/output schemas
- 93. Self-hosting docs UI static assets
- 95. OpenAPI callbacks
- 97. Pydantic Settings
- 99. Uvicorn workers
- 101. Running behind a proxy
- 103. HTTPS termination
- 105. Jinja2Templates
- 107. ORJSONResponse
- 109. Celery integration - <https://docs.celeryq.dev/>
- 54. Global dependencies
- 56. Dependencies in path decorator
- 58. Security() dependency
- 60. OAuth2 password flow
- 62. Current user dependency
- 64. Security scopes
- 66. Custom function middleware
- 68. BackgroundTasks
- 70. SQLAlchemy async session
- 72. Database model
- 74. Connection pooling
- 76. TestClient
- 78. Async test client
- 80. Test WebSocket
- 82. Static files
- 84. WSGIMiddleware
- 86. Metadata
- 88. Operation ID
- 90. Custom OpenAPI schema
- 92. Conditional OpenAPI by environment
- 94. Generate SDK clients
- 96. OpenAPI webhooks
- 98. Multiple environments
- 100. Docker deployment
- 102. Redis caching
- 104. FastAPI Cloud
- 106. GraphQL integration
- 108. API versioning (URL) - <https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design>
- 110. GraphQL integration - <https://strawberry.rocks/docs/integrations/fastapi>

Source Code & Repositories

- 111. Rate limiting
- 112. Request ID tracking
- 113. MessagePack serialization

Articles & Tutorials

- 114. api_route for multiple methods
- 116. Custom class middleware
- 118. ARQ for async tasks
- 120. Test file upload
- 122. Lifespan state shared with requests
- 124. Receive message
- 115. Client disconnect in a stream
- 117. Multiple background tasks
- 119. Alembic migrations
- 121. raise_server_exceptions
- 123. Class-based views
- 125. Gunicorn + Uvicorn - <https://uvicorn.dev/deployment/>